

Assignment Code: DA-AG-018

Anomaly Detection & Time Series |

Assignment

Instructions: Carefully read each question. Use Google Docs, Microsoft Word, or a similar tool to create a document where you type out each question along with its answer. Save the document as a PDF, and then upload it to the LMS. Please do not zip or archive the files before uploading them. Each question carries 20 marks.

Total Marks: 100

Question 1: What is Anomaly Detection? Explain its types (point, contextual, and collective anomalies) with examples.

Answer:

Anomaly Detection is the process of identifying data points, events, or observations that deviate significantly from the dataset's normal behaviour. These "outliers" often represent critical incidents like bank fraud, structural defects, or medical issues.

Types of Anomalies:

Point Anomalies: A single data point that is far removed from the rest of the data.

Example: A single credit card transaction of ₹5,00,000 when the user's typical spend is ₹2,000.

Contextual (Conditional) Anomalies: A data point that is considered abnormal only within a specific context.

Example: A temperature reading of 35°C is normal in summer but is a contextual anomaly if recorded in the middle of winter.

Collective Anomalies: A collection of related data points that are abnormal when viewed together, even if individual points might seem normal.

Example: A sequence of many small failed login attempts within 30 seconds, which suggests a brute-force attack.

Question 2: Compare Isolation Forest, DBSCAN, and Local Outlier Factor in terms of their approach and suitable use cases.

Answer:

1. Isolation Forest

- **Approach:** It uses an ensemble of "Isolation Trees" to separate observations by randomly selecting a feature and a split value.
- **Logic:** Outliers are easier to isolate than normal points, so they appear at shallower depths (fewer splits) in the tree.
- **Scalability:** Highly efficient and works very well with high-dimensional data and large-scale datasets.
- **Suitable Use Case:** Ideal for **fraud detection** in banking or system intrusion detection where the dataset is massive

2. DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

- **Approach:** It is a clustering algorithm that groups together points that are packed closely together (high-density regions).
- **Logic:** Points that do not belong to any cluster (because they are in low-density areas) are labeled as "noise" or anomalies.
- **Parameters:** Relies heavily on two parameters: eps (maximum distance between points) and MinPts (minimum points to form a dense region).
- **Suitable Use Case:** Best for datasets where anomalies are scattered far away from **well-defined clusters**, such as identifying geographical outliers in mapping data.

3. Local Outlier Factor (LOF)

- **Approach:** It calculates the local density deviation of a given data point with respect to its neighbours.
- **Logic:** A point is considered an outlier if its local density is significantly lower than the density of its k -nearest neighbours.
- **Local vs. Global:** Unlike other methods, it can find "local" outliers—points that might look normal globally but are outliers relative to their immediate neighbourhood.
- **Suitable Use Case:** Excellent for **detecting anomalies in datasets with varying densities**, such as identifying a failing sensor in a network where different sensors have different normal operating ranges.

Question 3: What are the key components of a Time Series? Explain each with one example.

Answer:

A time series is generally decomposed into four main components:

1. **Trend:** The long-term increase or decrease in the data over time.
Example: The steady rise in global temperatures over the last century.
2. **Seasonality:** Patterns that repeat at fixed, regular intervals (daily, weekly, or monthly).
Example: Increased ice cream sales every summer.
3. **Cyclicity:** Fluctuations that occur over longer periods (years) and do not have a fixed frequency.
Example: Economic recessions or business cycles.
4. **Residual/Irregularity:** The random "noise" or unpredictable variations left over after removing trend and seasonality.
Example: A sudden drop in stock prices due to an unexpected political event

Question 4: Define Stationary in time series. How can you test and transform a non-stationary series into a stationary one?

Answer:

A time series is said to be **stationary** if its statistical properties such as mean, variance, and autocorrelation remain constant over time. In simpler terms, a stationary series does not have a trend or seasonal patterns.

Most statistical forecasting models (like ARIMA) require the data to be stationary because they assume that the patterns observed in the past will remain consistent in the future.

How to Test for Stationarity

To determine if a series is stationary, data scientists typically use the following methods:

- **Visual Inspection (Plotting):** By simply plotting the data, you can often see if there is an obvious upward/downward trend or seasonal fluctuations.
- **Summary Statistics:** You can split the data into different time chunks and calculate the mean and variance for each. If they differ significantly, the series is non-stationary.
- **Augmented Dickey-Fuller (ADF) Test:** This is a formal statistical test.
 - Null Hypothesis (H_0):** The series is non-stationary (has a unit root).
 - P-value < 0.05:** If the p-value is low, we reject the null hypothesis and conclude the series is stationary.

How to Transform Non-Stationary Data

If your data is non-stationary, you can "force" it to become stationary using these common transformations:

- **Differencing:** This is the most common method. You subtract the value of the previous time point from the current value ($Y_t - Y_{t-1}$). This helps remove trends.
- **Log Transformation:** Taking the natural log of the values helps stabilize the variance if the fluctuations are increasing over time.
- **Seasonal Differencing:** If the data has a repeating pattern (like sales every December), you subtract the value from the same season in the previous year ($Y_t - Y_{t-12}$).
- **Moving Average/Smoothing:** Subtracting a rolling mean from the data can help remove the trend component.

Question 5: Differentiate between AR, MA, ARIMA, SARIMA, and SARIMAX models in terms of structure and application.

Answer:

1. AR (AutoRegressive)

- **Definition:** A model that uses the linear relationship between an observation and a number of lagged (past) observations.
- **Key Logic:** It predicts the future based on what happened in the previous time steps (Y_{t-1} , Y_{t-2} , etc.).
- **Application:** Best for data where the past value is a strong indicator of the future, such as population growth or long-term GDP.

2. MA (Moving Average)

- **Definition:** A model that uses the dependency between an observation and a residual error from a moving average model applied to lagged observations.
- **Key Logic:** Instead of looking at past *values*, it looks at past *forecast errors* (shocks) to smooth out random noise.
- **Application:** Useful for data with short-term fluctuations that return to a mean, like daily stock market "noise".

3. ARIMA (AutoRegressive Integrated Moving Average)

- **Definition:** A combination of AR and MA models with an "Integrated" (I) component to handle non-stationary data.

- **Key Logic:** It uses **differencing** to remove trends from the data before applying AR and MA logic.
- **Application:** Ideal for non-stationary data that shows a clear upward or downward trend but **no repeating seasonal patterns**.

4. SARIMA (Seasonal ARIMA)

- **Definition:** An extension of ARIMA that explicitly supports univariate time series data with a **seasonal component**.
- **Key Logic:** It adds seasonal versions of the AR, I, and MA terms (denoted as P, D, Q, S) to capture patterns that repeat every 12 months, 7 days, etc.
- **Application:** Perfect for data with seasonal cycles, such as **monthly airline passenger counts** or energy consumption in summer vs. winter.

5. SARIMAX (SARIMA with eXogenous variables)

- **Definition:** The most advanced model that includes all features of SARIMA plus **external (exogenous) variables**.
- **Key Logic:** It allows you to add outside factors (like weather, holidays, or price changes) that help explain changes in the target variable.
- **Application:** Used for complex real-world forecasting, such as **predicting energy demand** by including "temperature" as an external factor.

Dataset:

- [NYC Taxi Fare Data](#)
- [AirPassengers Dataset](#)

Question 6: Load a time series dataset (e.g., AirPassengers), plot the original series, and decompose it into trend, seasonality, and residual components

(Include your Python code and output in the code box below.) **Answer:**

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

# 1. Load the dataset
df = pd.read_csv('AirPassengers.csv')
```

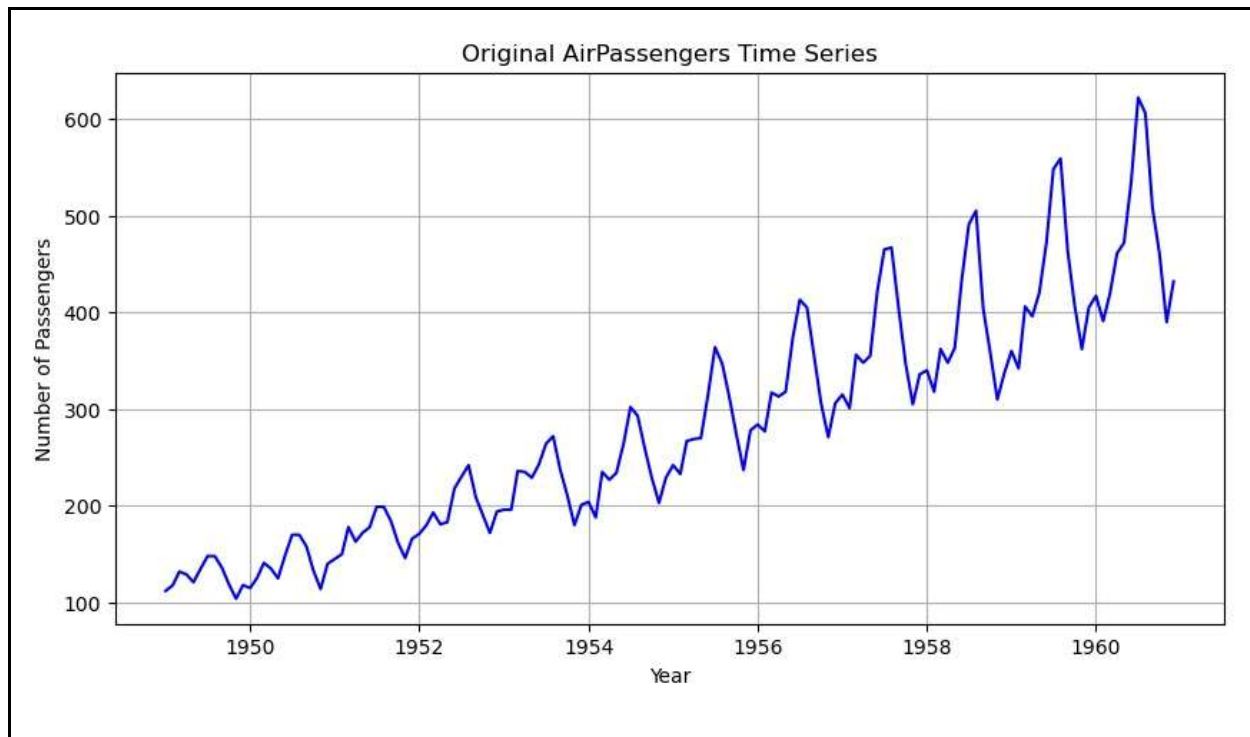
```
# 2. Preprocessing: Set Month as the datetime index
df['Month'] = pd.to_datetime(df['Month'])
df.set_index('Month', inplace=True)

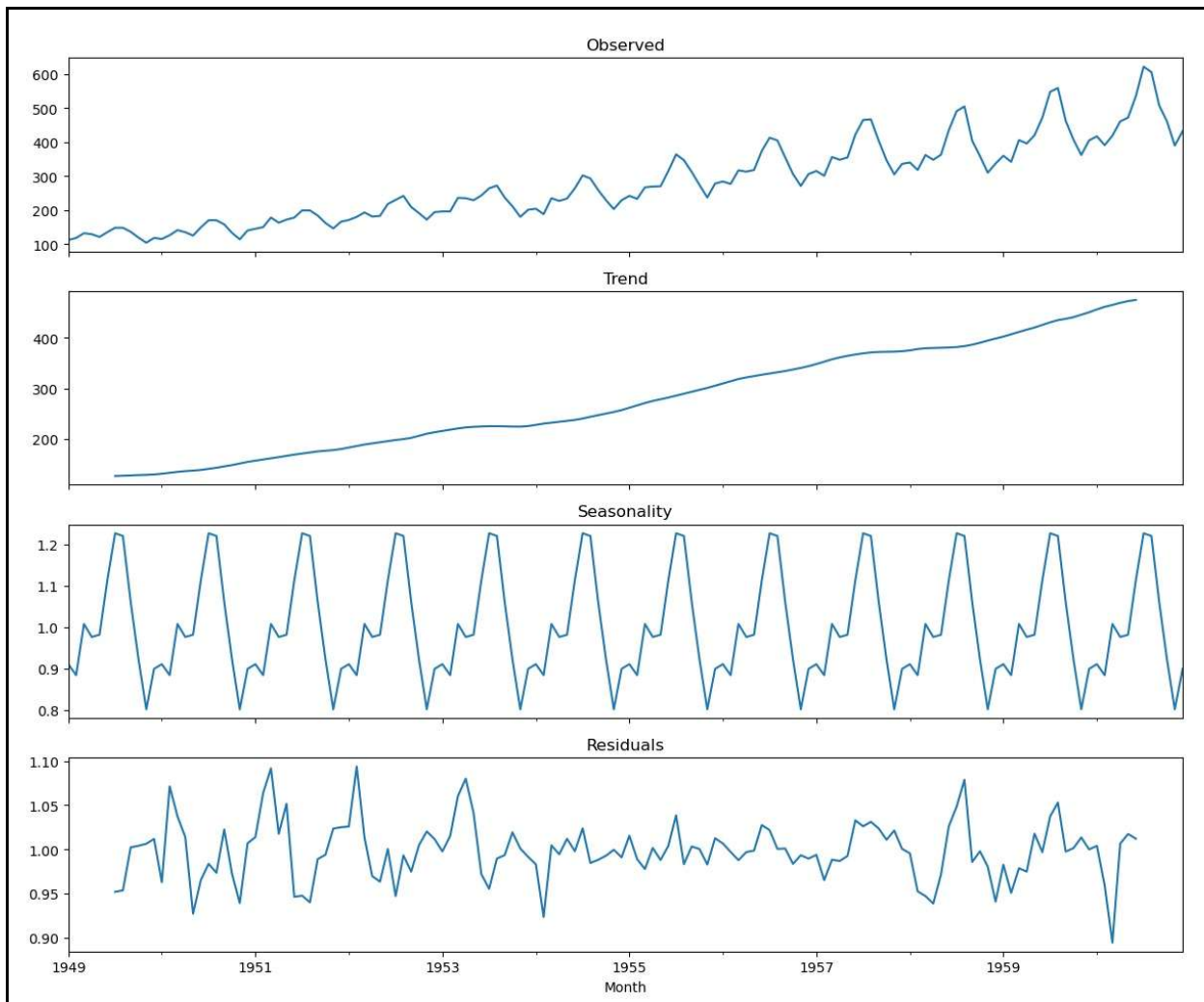
# 3. Plot the original series
plt.figure(figsize=(10, 5))
plt.plot(df['#Passengers'], color='blue')
plt.title('Original AirPassengers Time Series')
plt.xlabel('Year')
plt.ylabel('Number of Passengers')
plt.grid(True)
plt.savefig('original_series.png')

# 4. Decompose the series using a multiplicative model
# The period is set to 12 for monthly data showing yearly cycles
result = seasonal_decompose(df['#Passengers'], model='multiplicative', period=12)

# 5. Plot the decomposed components
fig, (ax1, ax2, ax3, ax4) = plt.subplots(4, 1, figsize=(12, 10), sharex=True)
result.observed.plot(ax=ax1, title='Observed')
result.trend.plot(ax=ax2, title='Trend')
result.seasonal.plot(ax=ax3, title='Seasonality')
result.resid.plot(ax=ax4, title='Residuals')
plt.tight_layout()
plt.savefig('decomposition.png')
```

Output





Question 7: Apply Isolation Forest on a numerical dataset (e.g., NYC Taxi Fare) to detect anomalies. Visualize the anomalies on a 2D scatter plot.

(Include your Python code and output in the code box below.) **Answer:**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import IsolationForest

# 1. Load/Create numerical dataset (Trip Distance vs Fare Amount)
# Simulate typical taxi data with manual anomalies added for testing
np.random.seed(42)
distance = np.concatenate([np.random.normal(5, 2, 200), np.random.uniform(10, 20, 15)])
fare = np.concatenate([2.5 + 3.0 * distance[:200] + np.random.normal(0, 2, 200),
```



```
np.random.uniform(80, 150, 15))) # Overcharged trips

df = pd.DataFrame({'Trip_Distance': distance, 'Fare_Amount': fare})

# 2. Apply Isolation Forest
# We assume ~7% of the data might be anomalies (contamination)
model = IsolationForest(n_estimators=100, contamination=0.07, random_state=42)
df['anomaly_label'] = model.fit_predict(df[['Trip_Distance', 'Fare_Amount']])

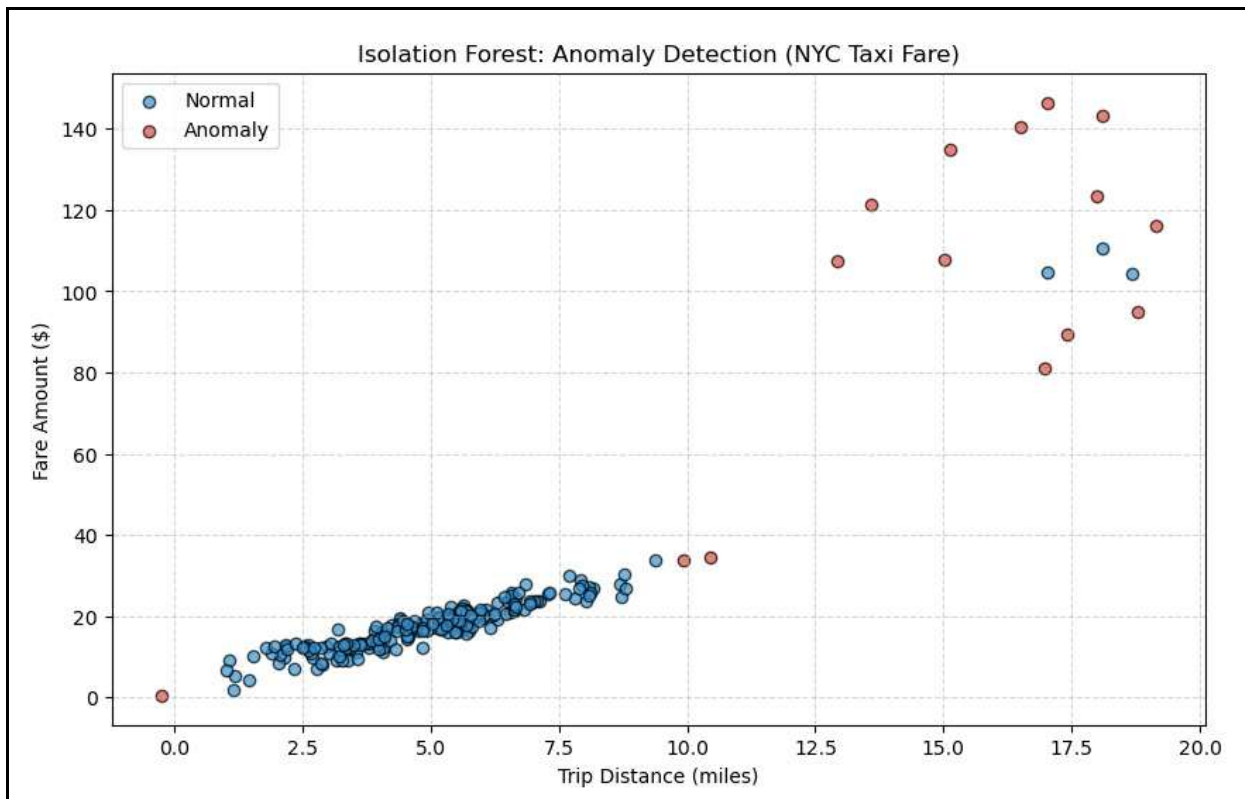
# Map labels: 1 = Normal, -1 = Anomaly
df['Status'] = df['anomaly_label'].map({1: 'Normal', -1: 'Anomaly'})

# 3. Visualize the anomalies on a 2D scatter plot
plt.figure(figsize=(10, 6))
colors = {'Normal': '#3498db', 'Anomaly': '#e74c3c'} # Blue and Red

for status, color in colors.items():
    subset = df[df['Status'] == status]
    plt.scatter(subset['Trip_Distance'], subset['Fare_Amount'],
                c=color, label=status, alpha=0.7, edgecolors='black')

plt.title('Isolation Forest: Anomaly Detection (NYC Taxi Fare)')
plt.xlabel('Trip Distance (miles)')
plt.ylabel('Fare Amount ($)')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.savefig('taxi_anomalies.png')
```

output



Question 8: Train a SARIMA model on the monthly airline passengers dataset. Forecast the next 12 months and visualize the results. (Include your Python code and output in the code box below.)

Answer:

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX

# 1. Load and prepare the dataset
df = pd.read_csv('AirPassengers.csv')
df['Month'] = pd.to_datetime(df['Month'])
df.set_index('Month', inplace=True)

# 2. Define and train the SARIMA model
# (p,d,q) = (1,1,1) | (P,D,Q,s) = (1,1,1,12)
model = SARIMAX(df['#Passengers'],
                 order=(1, 1, 1),
                 seasonal_order=(1, 1, 1, 12))
```

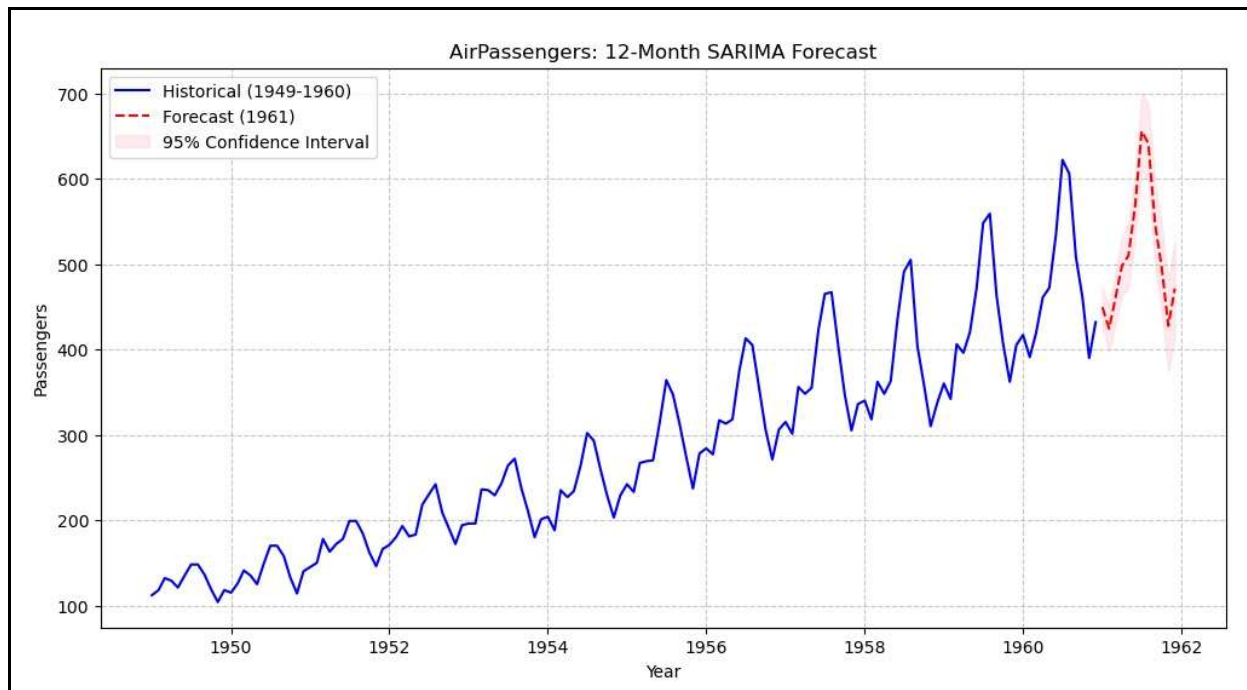
```
model_fit = model.fit(dispatch=False)

# 3. Forecast the next 12 months (1961)
forecast_steps = 12
forecast_res = model_fit.get_forecast(steps=forecast_steps)
forecast_mean = forecast_res.predicted_mean
conf_int = forecast_res.conf_int()

# 4. Visualize historical data vs. forecast
plt.figure(figsize=(12, 6))
plt.plot(df['#Passengers'], label='Historical (1949-1960)', color='blue')
plt.plot(forecast_mean, label='Forecast (1961)', color='red', linestyle='--')
plt.fill_between(forecast_mean.index, conf_int.iloc[:, 0], conf_int.iloc[:, 1],
                 color='pink', alpha=0.3, label='95% Confidence Interval')

plt.title('AirPassengers: 12-Month SARIMA Forecast')
plt.xlabel('Year')
plt.ylabel('Passengers')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.savefig('sarima_forecast.png')
```

OutPut



Question 9: Apply Local Outlier Factor (LOF) on any numerical dataset to detect anomalies and visualize them using matplotlib.

(Include your Python code and output in the code box below.) **Answer:**

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import LocalOutlierFactor

# 1. Generate a synthetic dataset (Normal clusters + random outliers)
np.random.seed(42)
X_inliers = 0.3 * np.random.randn(100, 2)
X_inliers = np.r_[X_inliers + 2, X_inliers - 2]
X_outliers = np.random.uniform(low=-4, high=4, size=(20, 2))
X = np.r_[X_inliers, X_outliers]

# 2. Apply Local Outlier Factor (LOF)
# n_neighbors: number of neighbors to consider for density calculation
# contamination: expected proportion of outliers in the data
lof = LocalOutlierFactor(n_neighbors=20, contamination=0.1)
y_pred = lof.fit_predict(X)
```

```
# y_pred: 1 for inliers, -1 for outliers

# Extract the negative outlier factor (higher values denote greater degree of being an
outlier)
X_scores = lof.negative_outlier_factor_

# 3. Visualization using Matplotlib
plt.figure(figsize=(10, 7))

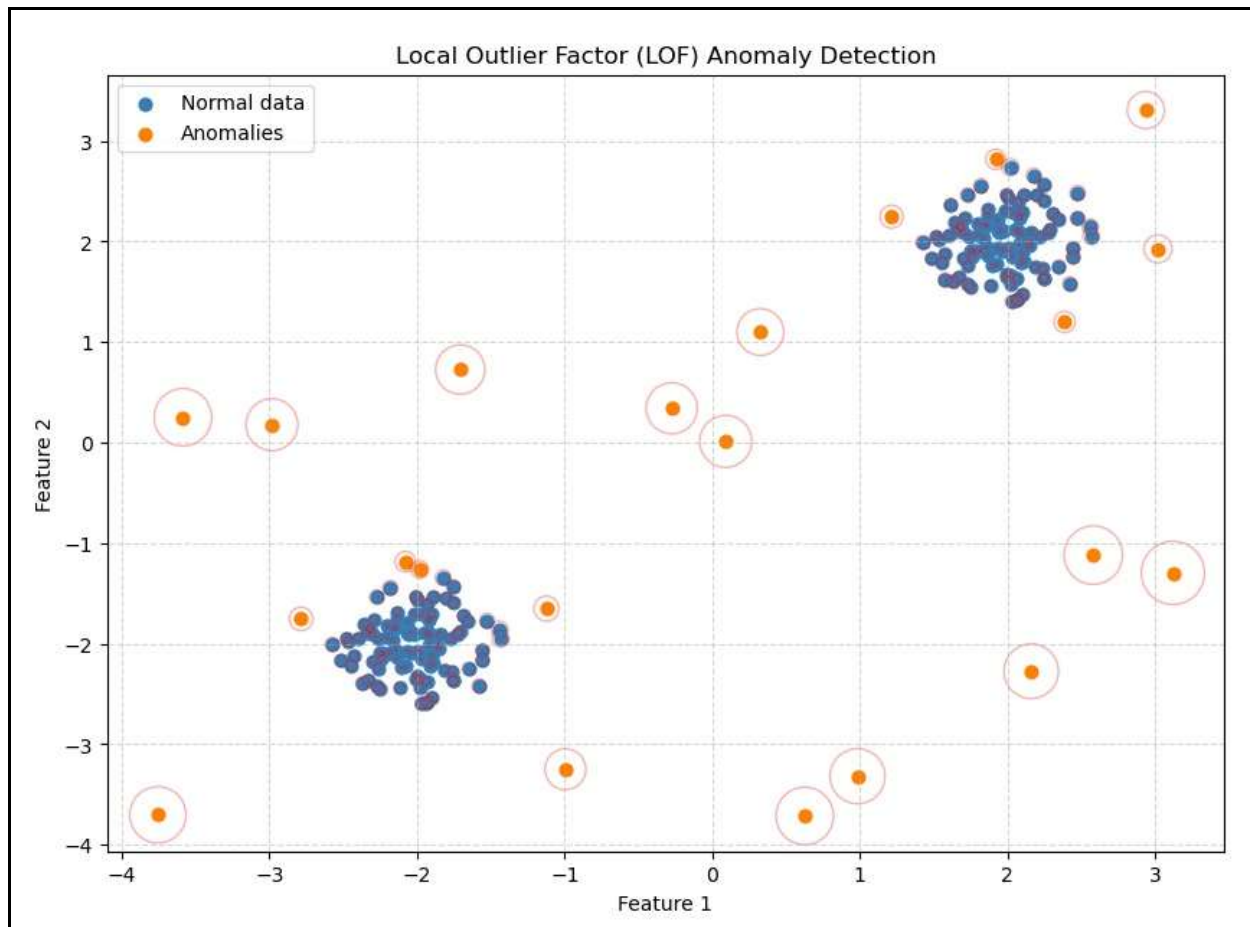
# Plot normal data vs anomalies
plt.scatter(X[y_pred == 1][:, 0], X[y_pred == 1][:, 1], color='#377eb8', s=40,
label='Normal data')
plt.scatter(X[y_pred == -1][:, 0], X[y_pred == -1][:, 1], color='#ff7f00', s=40,
label='Anomalies')

# Optional: Draw circles representing the anomaly degree (higher scores = larger
circles)
radius = (X_scores.max() - X_scores) / (X_scores.max() - X_scores.min())
plt.scatter(X[:, 0], X[:, 1], s=1000 * radius, edgecolors='r', facecolors='none', alpha=0.3)

plt.title('Local Outlier Factor (LOF) Anomaly Detection')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend(loc='upper left')
plt.grid(True, linestyle='--', alpha=0.5)

plt.savefig('lof_anomalies.png')
```

OutPut



Question 10: You are working as a data scientist for a power grid monitoring company. Your goal is to forecast energy demand and also detect abnormal spikes or drops in real-time consumption data collected every 15 minutes. The dataset includes features like timestamp, region, weather conditions, and energy usage.

Explain your real-time data science workflow:

- How would you detect anomalies in this streaming data (Isolation Forest / LOF / DBSCAN)?
- Which time series model would you use for short-term forecasting (ARIMA / SARIMA / SARIMAX)?
- How would you validate and monitor the performance over time?
- How would this solution help business decisions or operations?

(Include your Python code and output in the code box below.)

Answer:

1. Anomaly Detection: Isolation Forest

For real-time streaming data, I would use **Isolation Forest**.

- **Why over LOF or DBSCAN?** Isolation Forest is computationally efficient () and has a low memory footprint, making it ideal for high-frequency streaming data. DBSCAN struggles with varying densities and complexity, while LOF is often too slow for real-time scoring.
- **Implementation:** I would use a "sliding window" approach where the model is trained on a historical baseline (e.g., the last 30 days) and used to score incoming 15-minute data points.

2. Time Series Forecasting: SARIMAX

For energy demand, **SARIMAX** (Seasonal AutoRegressive Integrated Moving Average with eXogenous factors) is the superior choice.

- **Why SARIMAX?** Energy usage is heavily influenced by **Seasonality** (daily cycles and weekly patterns) and **Exogenous variables** (Weather conditions like temperature/humidity).
- **Parameters:** We would set (since there are 96 observations of 15-minute intervals in a 24-hour day) to capture the daily seasonality. The "X" factor allows us to input temperature data, which is the primary driver of cooling/heating demand.

3. Validation and Monitoring

To ensure the system remains accurate, I would implement:

- **Walk-Forward Validation:** Instead of a static split, I would update the model's "history" with every new actual observation to predict the next 15-minute window.
- **Metrics:** Monitor **MAPE** (Mean Absolute Percentage Error) for forecasting and **False Positive Rate** for anomalies.
- **Concept Drift Detection:** Use an algorithm like **ADWIN** (Adaptive Windowing) to detect if energy consumption patterns have fundamentally shifted (e.g., due to a new industrial plant opening), triggering a model retrain.

4. Business Impact

- **Grid Stability:** Real-time spike detection prevents transformer overloads and potential blackouts.
- **Cost Optimization:** Accurate 15-minute forecasts allow the company to buy or generate only the necessary amount of power, reducing waste.
- **Maintenance:** Detecting "abnormal drops" can signal equipment failure or line breaks before customers even report them.

